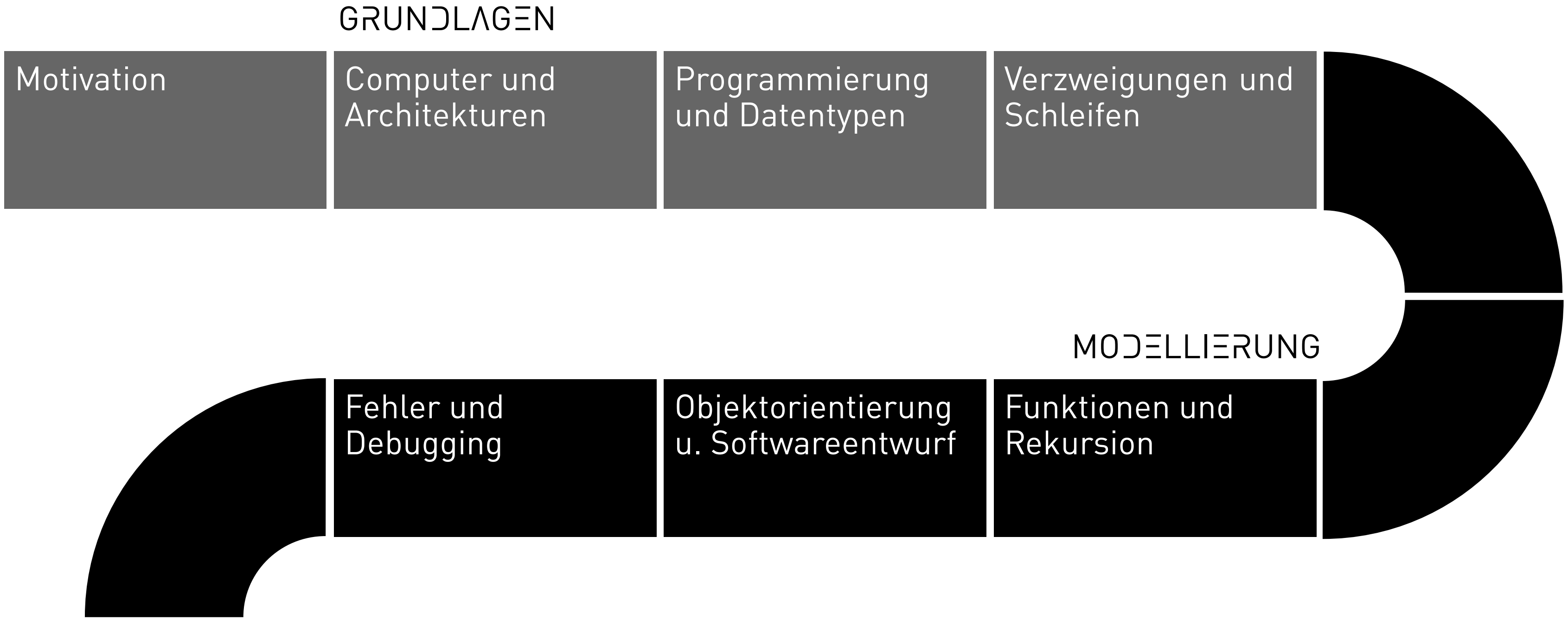
Programming and databases

Joern Ploennigs

Branching

MIDJOURNEY: RED OR BLUE PILL, REF. LEONARDO DA VINCI

Process


GRUNDLAGEN
Motivation
Computer und Architekturen
Programmierung und Datentypen
Verzweigungen und Schleifen
MODELLIERUNG
Funktionen und Rekursion
Objektorientierung u. Softwareentwurf
Fehler und Debugging

Conditional Branching

- We can use control statements that, based on a condition, branch to a particular statement.
- These conditions are statements whose output is a boolean value (True or False):
  - `True` - Boolean values
  - `a` - Variable values, if a has been assigned a boolean, i.e., the boolean equivalent of a
  - `2 < 5` - All comparison operators

If ... Then ...

- The basic form of such branching is:

> *"If the condition is true, then execute the following statements."*

- There is often also an "else" that executes the following statement instead.
- Python has three branching statements:
  - `if` - if
  - `else` - else
  - `elif` - short for else-if

If Statement

- The if statement is needed to start a conditional branch.
- Important: The conditional code block must be indented

```python
if Condition:
    # then execute the following block only if the condition is true
    Statement1
    Statement2
    # ...
```

If the condition is `True`, the indented statements will be executed.

Example: Using an if statement to skip code

```python
activateOutput = True
a = 10
b = 13
c = math.sqrt(a**2 + b**2) # Pythagoras

if activateOutput:
    print("Seitenlänge:" + str(c))
    umfang = a + b + c
    # ...
```

Alternatives

- `else` follows only after an `if`
- Is executed when the condition specified in the `if` evaluates to false
- Also requires indentation of the conditional code block

```
if Condition:
    # then execute the following block only if the condition is true
    Statement1
    Statement2
    # ...
else:
    Statement1b
    # ...
```

Example: Catching Division by 0 (Simple Branching)

Implicit Version

```python
a = 5
b = 0
if b:  # the truth value of an int is False for 0 and True for != 0
    c = a / b
else:
    print("Division by Zero")
    c = None
```

Example: Catching division by zero (simple branching)

Explicit version

```python
a = 5
b = 0
if b != 0: # Explicit truth value
    c = a / b
else:
    print("Division by Zero")
    c = None
```

Nested Conditionals

By indenting multiple levels, we can define nested conditionals:

```
if Condition1:
    StatementA
else:
    if Condition2:
        StatementB
    else:
        StatementC
```

Example: Checking the sign (Nested branching)

```python
if a < 0:
    print("Number is negative")
else:
    if a > 0:
        print("Number is positive")
    else:
        print("Number is zero")
```

Multiple branching: ELIF statement

- Instead of an `else` statement, we can use the `elif` statement, which, like `if`, has a condition.
- An `elif` statement can be followed by:
  - Another `elif` statement
  - `else` statement - is executed when none of the previous statements were true
  - No statement – second variant of the optional code

Example: Testing the sign (multi-way branching)

```python
if a < 0:
    print("Number is negative")
elif a > 0:
    print("Number is positive")
else:
    print("Number is zero")
```

Much cleaner and more readable than nested if-else statements!

Many Alternatives

In Python 3.10, the match statement was introduced, which already exists in many other languages. It allows you to compare a variable against several possible values.

Traditionally (many elifs):

```
if Variable == ValueA:
    StatementA
elif Variable == ValueB:
    StatementB
else:
    StatementC
```

New (match-case):

```
match Variable:
    case ValueA:
        StatementA
    case ValueB:
        StatementB
    case _:
        StatementC
```

Lesson Learned

©Midjourney: Six Sense

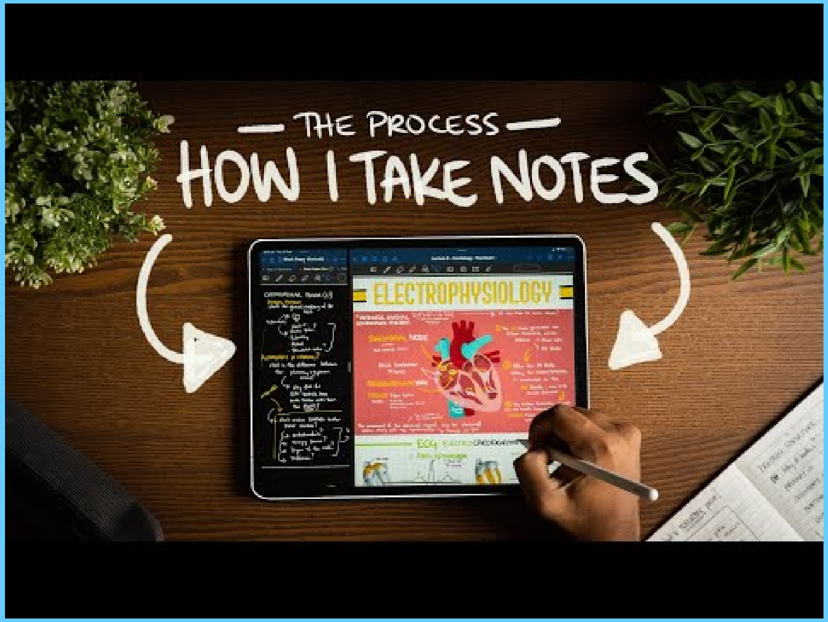
What learning styles are there?

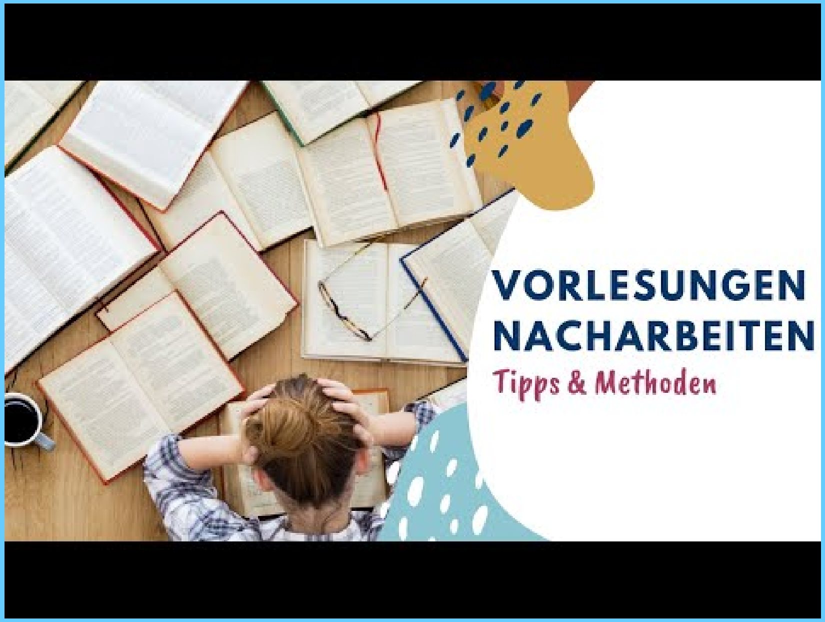
Lesson Learned - Notes

- You should always take notes (because we remember written material better, both by touch and visually).
- You should not copy the slides, but rather capture key information, terms, and questions.
- You should take the time to review your notes, clarify questions, and structure the content yourself.

Follow-Up

- **S**urvey (Gain an overview): Skim through a topic to gain an overview
- **Q**uestion (Questions): Note down questions about points you don't understand
- **R**ead (Read): Read through your notes; highlight keywords; research the questions
- **R**ecite (Recall): Recapitulate each section (content, keywords, relationships). Organize the content, e.g., with mind maps. Repeat what you've learned multiple times (flashcards)
- **R**eview (Recap): Mentally review the key points and answer your questions





Questions?